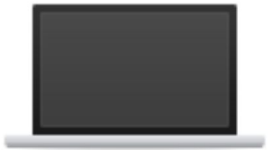
Introduction to Git and GitHub

High Level Programming

Alberto Zucchetta

Working, saving and sharing code

- Simplest case: “I work alone, on a single computer” - probably most of you, until today



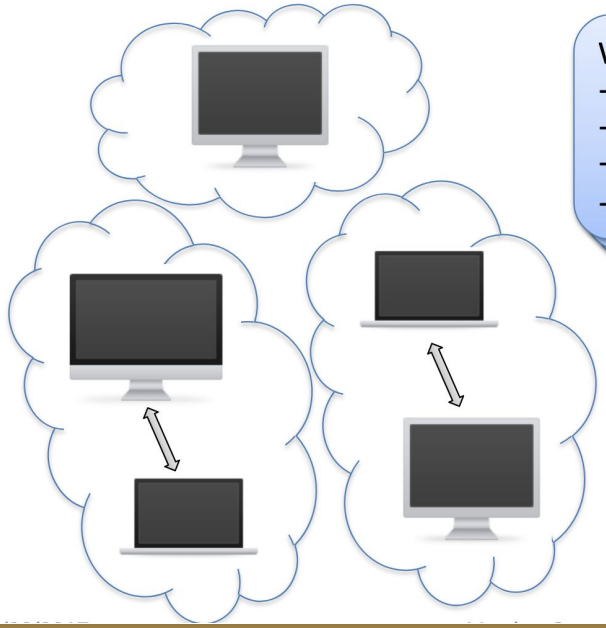
What I need:
- backup;
- different saved versions;
- early and frequently saving.

What I can use:
- external hard drives;
- dedicated folder;
- Dropbox folder;
- …

What if…
- … I forget to save a specific version and then I need it?
- … I delete/loose a previous version?

Working, saving and sharing code

- More complex: “I work in a team” - you, in this course



What I need:
- backup;
- different saved versions;
- early and frequently saving;
- shared conventions on file names.

What I can use:
- USB memory sticks;
- external hard drives;
- Dropbox folder;
- e-mails;
- …

What if…
- … a team member forgets to save a specific version and then someone else needs it?
- … someone deletes/looses a version?
- … someone forgets to share a specific version of the projects?

Other issues:
- who has the latest version?
- who has the right to edit?
- how to ensure that everyone sees up-to-date versions of everything?
- how to handle conflicts?

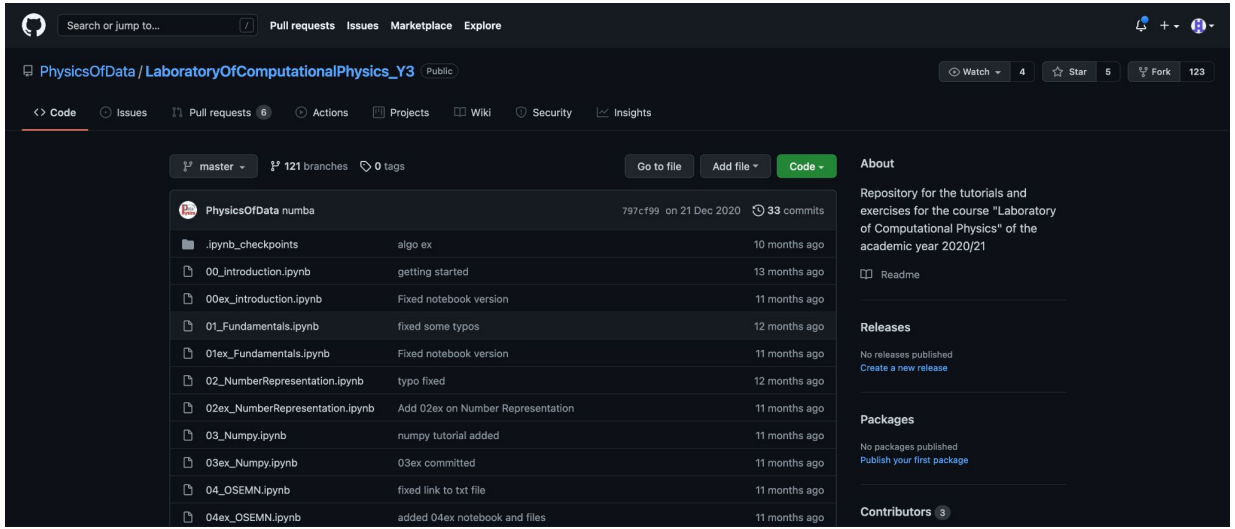
What is Git, and why it's important

Git is an Open Source **Distributed Version Control System:**

- **Control System**: means that Git is a content tracker - but it also provides a lot of other functionalities
- **Version Control System**: the code which is stored in Git keeps changing as more code is added. Also, many developers can add code in parallel. So Version Control System helps in handling this by maintaining a history of what changes have happened. Also, Git provides features like *branches* and *merges*
- **Distributed Version Control System**: Git has a remote repository which is stored in a server and a *local repository* which is stored in the computer of each developer. This means that the code is not just stored in a central server, but the full copy of the code is present in all the developers' computers.

Git and GitHub

- GitHub is a code-hosting service, extremely well integrated with git
- Can be reached at github.com
- A commercial company:
  - a free account can host countless open source projects
  - more advanced features, like private repositories, require a subscription fee



Basic Concepts



Repository:

- place where you store all your work
- contains every version of your work that has ever existed:
  - files
  - directories layout
  - history
- can be shared with multiple users
- can be created from scratch with the command
  - `$ git init`
  - `$ git branch -M main`
  - `$ git remote add origin git@github.com:<user>/<repo name>.git`
  - `$ git push -u origin main`
- can be cloned from an existing repository (our case, more frequent)
  - `$ git clone <url>`

Basic Concepts



Working copy:

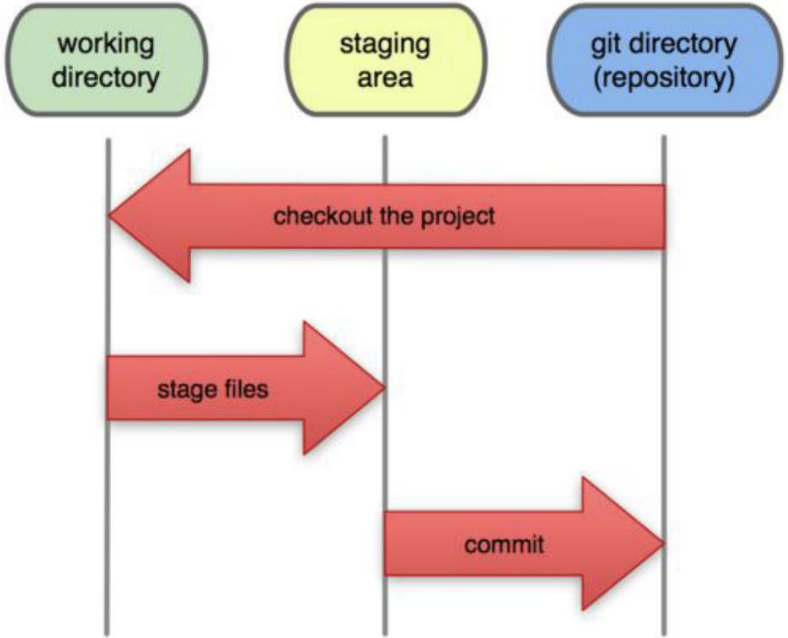
- a local copy of the repository used for... working
- it also contains some metadata so that it can keep track of the state of things
  - has a file been modified?
  - is this file new?
  - has a file been deleted?
- by default, new files in the working directory are *untracked*, so by default git does not care about them unless you tell it so
- every time a file is modified, and we want to propagate those changes to the repository, we have to tell git to consider (*track*) the file and add it to the *staging area*:

```
$ git add <file1> <file2> <...>
```

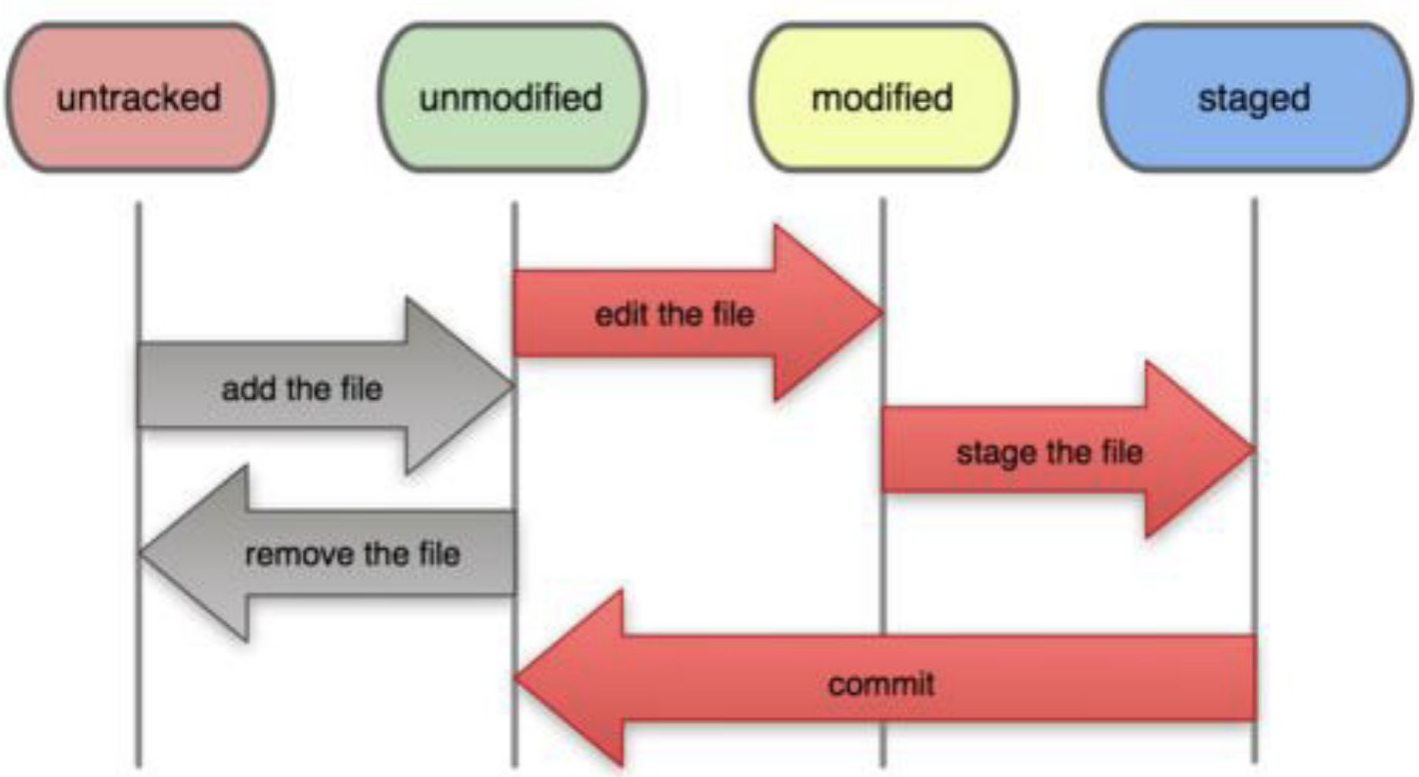
Local operations

A local project has 3 areas:

1. the working directory on your own pc
2. a staging area, still on your pc, which groups all files that have been modified and are about to be committed to the repository
3. the repository, on cloud server



The file status lifecycle



Local operations

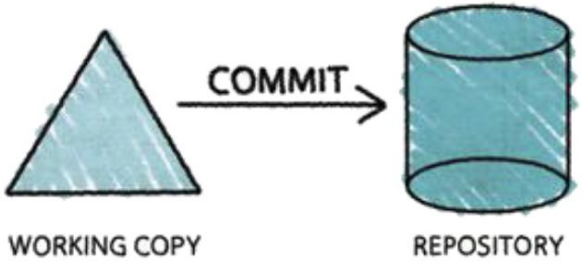
- Git helps you to check what is the status of the repository, giving an overview of the files in the local directory, and specifically
  - which files are tracked or not
  - which files have been modified

```
$ git status
```

- It’s also very helpful to check which changes have been done to the local files that have not been staged:

```
$ git diff
```

Basic Concepts

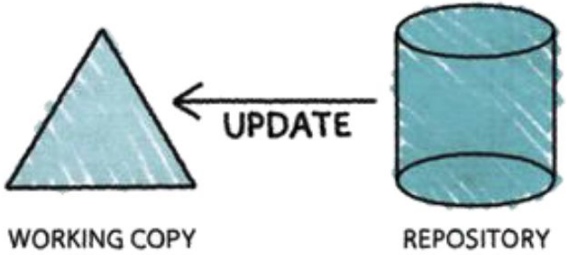


Commit:

- the operation that propagates the changes to the files in the staging area to the repository
  - atomically performed by modern version control tools
- it is mandatory to provide a log message (or comment) when you commit to explain the changes you have made
  - the message becomes part of the history of the repository
- the git command to run the commit is

```
$ git commit -m "meaningful commit message"
```

Basic Concepts



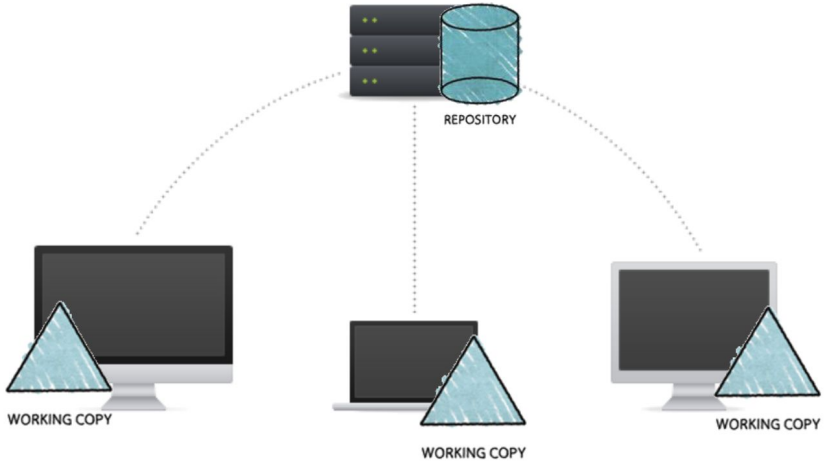
Update:

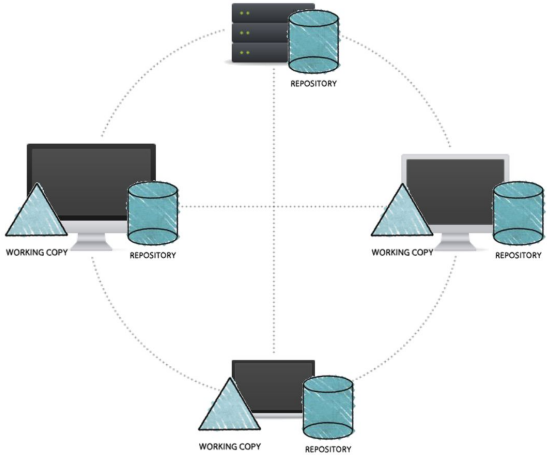
- update the working copy with respect to the repository
  - apply changes from the repository
  - merge such changes with the ones you have made to your working copy, if necessary

Centralized vs Distributed Version Control

Centralized:

- one central repository
- exclusive client-server relationship

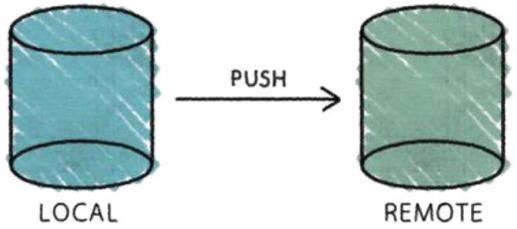




Distributed:

- clients and server have the full copy of the repository
  - local repositories clone a remote repository
- possible to have more than one server:
  - several instances of the same repository

Basic Concepts

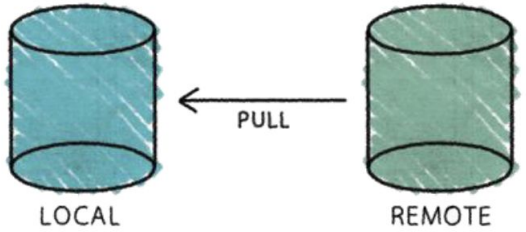


Push:

- copy changes from a remote repository instance to a **remote** one
- synchronization between two repository instances
- command:

```
$ git push <name of the remote> <branch>
```

Basic Concepts



Pull:

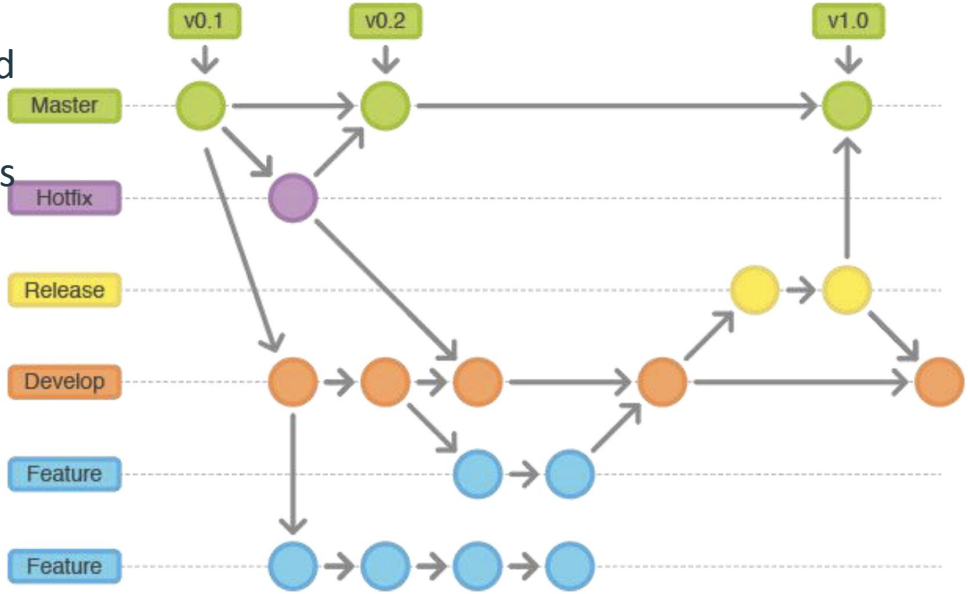
- copy changes from a remote repository instance to a **local** one
- synchronization between two repository instances
- command:

```
$ git pull
```

- make sure to run this command before working on a local file!

The branching model

- A **branch** is an image of the repository used to develop new features
  - in our case, every student will have its own branch
- Branching means you diverge from the main line of development and continue to do work without messing with that main line
- The default branch is usually named **main**



See also:
https://git-scm.com/book/it/v2/Git-Branching-Branches-in-a-Nutshell

The branching model

- To create a new branch:

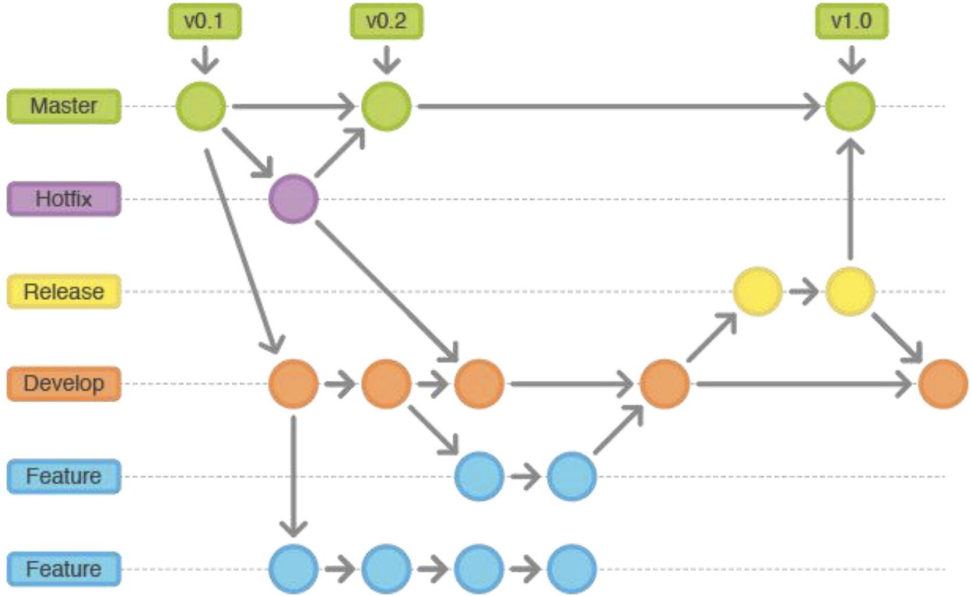
  ```
  $ git checkout -b
  <branchname>
  ```

- To switch to an existing branch:

  ```
  $ git checkout <branchname>
  ```

- To switch back to the default branch:

  ```
  $ git checkout main
  ```

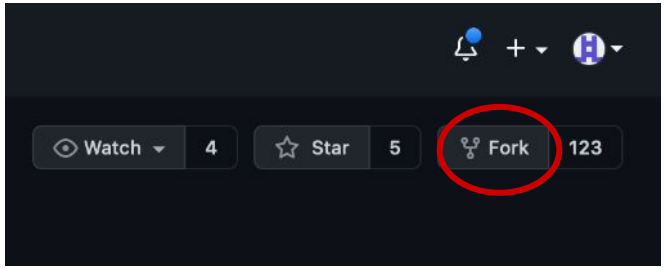


Shortlist of the most used commands

command	description
`git clone url [dir]`	copy a git repository so you can add to it
`git add files`	adds file contents to the staging area
`git commit`	records a snapshot of the staging area
`git status`	view the status of your files in the working directory and staging area
`git diff`	shows diff of what is staged and what is modified but unstaged
`git help [command]`	get help info about a particular command
`git pull`	fetch from a remote repo and try to merge into the current branch
`git push`	push your new branches and data to a remote repository
others: `init`, `reset`, `branch`, `checkout`, `merge`, `log`, `tag`	

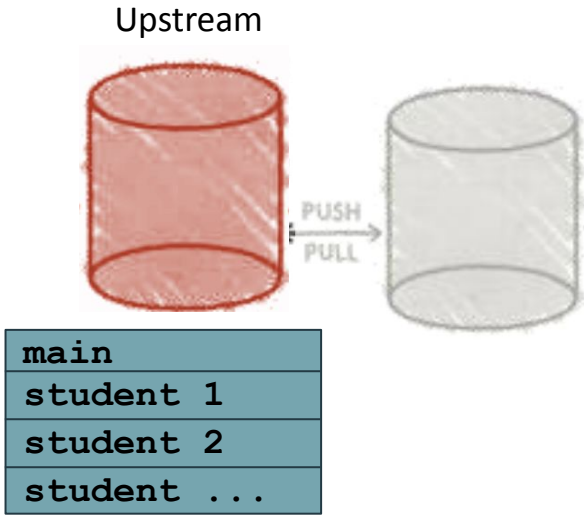
The Fork and Pull model

- There are two primary ways people collaborate on GitHub:
  a. Shared repository
  b. Fork and Pull
- Any user can “fork” an existing repository, creating a copy of the *upstream* repository to their personal account
- Forking can be done on GitHub, by clicking on the “fork” button on every public repository page:



- No permission is needed by the upstream repository, and users have full control of their forked repository and are free to modify the content
- However, the inverse is not true, as there is a specific procedure (called *Pull Request*) to propagate the changes from the remote to the upstream repository

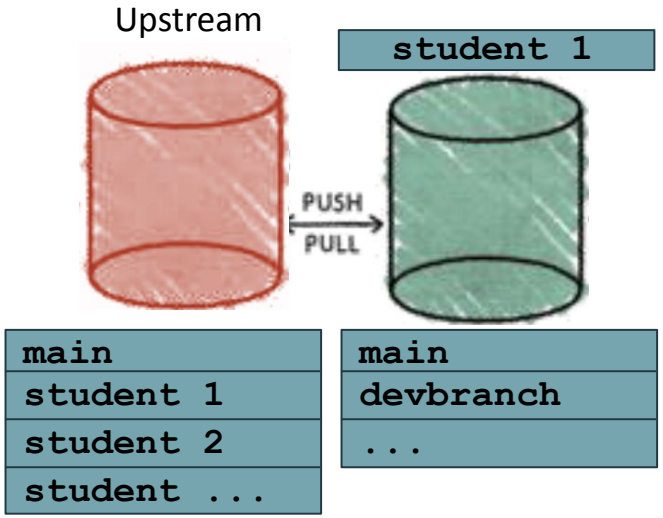
The upstream repository



Upstream repository:

- Unique copy: in our case, where I will add the material of the course week by week
- Will have one branch for each student, named NameSurname
- You will push your exercises to **your branch, not the main**

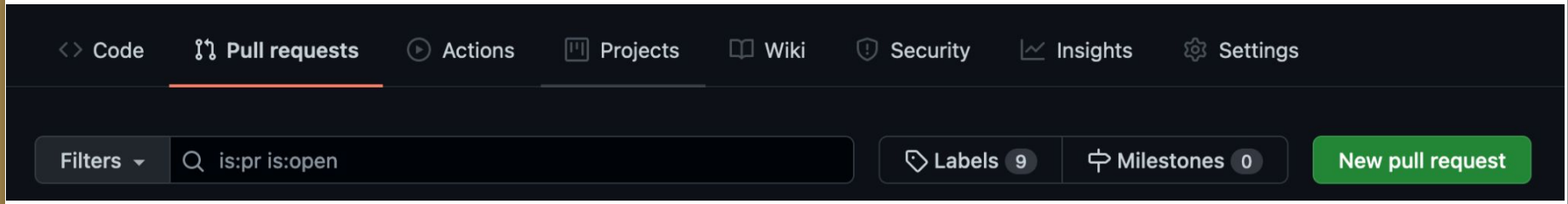
The upstream repository



- With the fork, you will create your remote repository
- The write access to the upstream is sometimes (and in our case) disabled
  - that means that the users (the students) need an explicit approval by the owners of the upstream repository to propagate the changes
- The merging of two branches from two different repositories is called **pull request**
  - you will need to do it to “hand over” the exercises

Pull requests

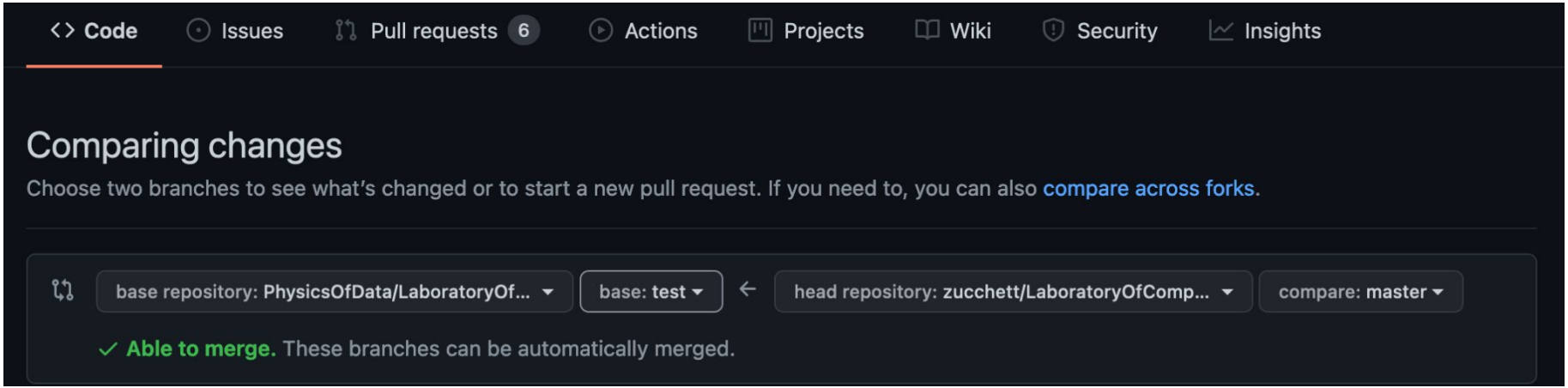
- This operation is usually done on GitHub, but it can be done also via command line
- Make sure via the web interface that you pushed all changes to your local repository
  - further changes may require another PR
- Click on the *Pull requests* tab
- Create the PR by clicking the big green button



https://docs.github.com/en/enterprise-server@3.0/github/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/creating-a-pull-request

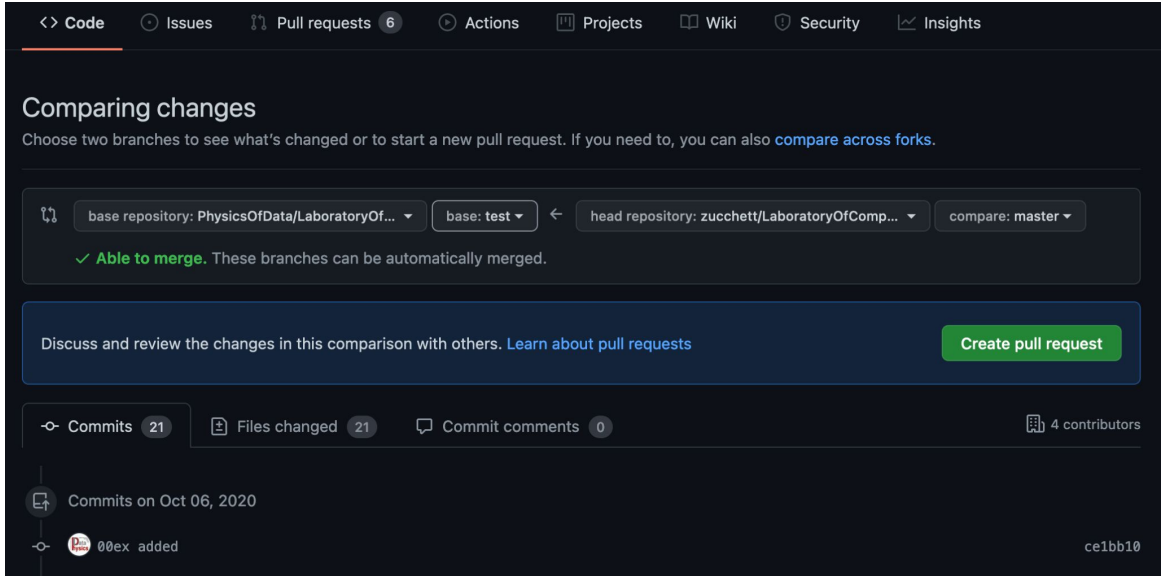
Pull requests

- Check and set the starting branch (your own development branch)
- Set the target branch on the upstream repository
  - the target branch **is not the main**, it's the one that has your name
  - PRs to the master will be quickly rejected!
- A confirmation "**Able to merge**" will indicate that there are no conflicts, and the PR can potentially be successful



Pull requests

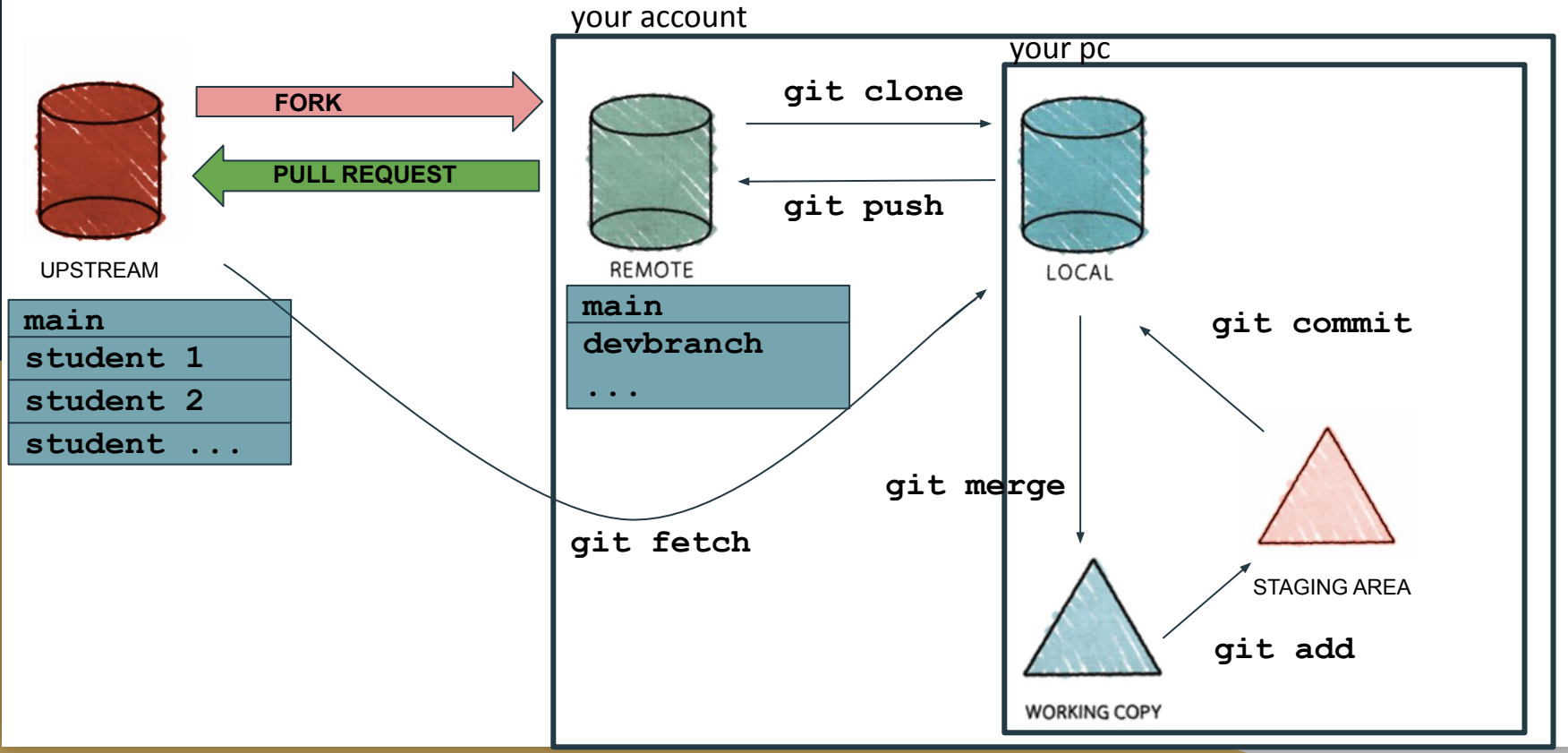
- GitHub automatically provides a summary of the files that are about to be pushed
  - check carefully that this is intended - this is the last step where it can be done
- Click on the green button to “Create pull request”
- Your PR will appear on the “Pull requests” tab on the remote repository



Pull requests

- Wait for approval. It is usually not immediate, and take hours, days, weeks… because the owners of the remote repository have to inspect the content of the PR
- Comments can be posted to require changes to a specific PR on the GitHub interface
  - you'll likely be notified by e-mail
  - if it's the case, post an answer and act as requested
- In case further modifications are needed, you should push your changes to your local branch from which the PR originates. The PR is automatically updated, **you don't need to open a new one**
- The date and time of the original PR is what counts for the deadline, unless there is something obvious...

Workflow summary



Installing git

- Check if git is already installed:
  - open a terminal and type “git”

- Windows:
  - download and install Git from http://git-scm.com/downloads
- Linux:
  - install it from your package manager or via http://git-scm.com/downloads
- Mac:
  - install it from http://git-scm.com/downloads

Git setup

- Create your account on github.com
- Unfortunately, since August, GitHub requires a new authentication procedure that requires the generation of a personal token:
  - follow this step-by-step guide: https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/creating-a-personal-access-token
  - this has to be done only once, if you clone the repo with your token
  - Remember to copy the token to a safe place! Once you have created it, you won't be able to see it again. If you lose it, you have to generate a new one.
- On your pc, and after cloning the repository, set the username and the e-mail that will be used to "sign" the commits:
  - **`$ git config --global user.name "Name Surname"`**
  - **`$ git config --global user.email namesurname@mail.com`**
- You can check that they are properly set by using **`$ git config -list`**
- If you omit **`-global`**, you can specify different settings for each repository

The to-do list

To be done only the first time:

1. Install git and create a GitHub account
2. Generate your token on GitHub, can copy it to a safe place. Follow this guide: https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/creating-a-personal-access-token
3. Got to GitHub to the upstream repository webpage and fork it: https://github.com/zucchett/HighLevelProgramming2021
4. On your pc, clone your forked repository:
   a. `$ git clone https://<YourToken>@github.com/<YourUsername>/HighLevelProgramming2021.git`
5. Point your forked repository to the upstream repository:
   a. `$ git remote add-upstream https://github.com/zucchett/HighLevelProgramming2021.git`
6. Fetch the updates directly from upstream:
   a. `$ git fetch upstream`
7. Optional: create your own development branch to edit the code that tracks your branch on the upstream:
   a. `$ git checkout -b <your branch name> upstream/<NameSurname>`

The to-do list

Development cycle:

1. Before starting any development, check whether the original repository has been updated, and merge the changes into your main:
   a. `$ git checkout main`
   b. `$ git fetch upstream`
   c. `$ git merge upstream/main`
2. In case your PR has been recently approved, and/or you have a development branch, synch it:
   a. `$ git checkout <your branch name>`
   b. `$ git fetch upstream/<NameSurname>`
   c. `$ git merge upstream/<NameSurname>`
3. Get the updates from the master:
   a. `$ git merge main`
4. Develop code
5. Add modified files to the staging area:
   a. `$ git add <file1> <file2>`
6. Commit changes to your local repository:
   a. `$ git commit -m "some message"`
7. Push local changes to your remote repository:
   a. `$ git push origin <your branch name>`
8. When it's time, prepare and open the Pull Request

Troubleshooting

- **Problem:** the push is rejected with a similar error message

```
To http://centralserver.com/myproject

! [rejected]        master -> master (non-fast-forward)

error: failed to push some refs to
'http://centralserver.com/myproject'
```

- **Cause**: very common error caused by misalignment between the working directory (usually out-to-date) and the repository
  - somebody else pushed a new version of the file you're working on, and you are modifying and trying to push the old version
- **Solution**: do a pull first, implement the changes, then push again
- **Alternative**: push with conflicts and solve them (see documentation)

Troubleshooting

- Git is not perfect, and some commands and error messages could be difficult to understand or counter-intuitive
- Before asking, please try to solve the problem yourself
- It’s very likely that googling the error message will point you to the right direction
- If anything else fails, as a last resort:



https://xkcd.com/1597/

Some references and credits

- Git handbook (start from here):
  - https://guides.github.com/introduction/git-handbook/
- Git cheat sheet:
  - https://education.github.com/git-cheat-sheet-education.pdf
- Branching in a nutshell:
  - https://git-scm.com/book/it/v2/Git-Branching-Branches-in-a-Nutshell
- Working with forks:
  - https://docs.github.com/en/github/collaborating-with-pull-requests/working-with-forks
- Pull requests:
  - https://docs.github.com/en/enterprise-server@3.0/github/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/creating-a-pull-request
- Introduction slides:
  - https://elite.polito.it/files/courses/01QZP/2017/slide/Git-01-intro.pdf
  - https://alessandropellegrini.it/didattica/2019/ia/git-introduction.pdf